The Journal of Systems and Software 223 (2025) 112365

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Towards cost-efficient vulnerability detection with cross-modal adversarial reprogramming☆

Zhenzhou Tian [a,b,c,*], Rui Qiu [a], Yudong Teng [a], Jiaze Sun [a], Yanping Chen [a,b,c], Lingwei Chen [d]

[a] School of Computer Science and Technology, Xi'an University of Posts and Telecommunications, Xi'an, Shaanxi 710121, China
[b] Shaanxi Key Laboratory of Network Data Analysis and Intelligent Processing, Xi'an, Shaanxi 710121, China
[c] Xi'an Key Laboratory of Big Data and Intelligent Computing, Xi'an, Shaanxi 710121, China
[d] Department of Computer Science and Engineering, Wright State University, Dayton, 45435, OH, USA

ARTICLE INFO

Keywords:
Vulnerability detection
Adversarial reprogramming
Cross modal
Data-limited training

ABSTRACT

While deep learning has advanced the automatic detection of software vulnerabilities, current DL-based methods still face two major obstacles: the scarcity of vulnerable code samples and the high computational cost of training models from scratch, which, however, have been largely overlooked. This paper introduces Capture, a novel Cross-modal Adversarial rePrograming approach Towards cost-efficient vUlneRability dEtection, which reduces the need for well-labeled large vulnerable datasets and minimizes training time. Specifically, Capture first performs lexical parsing and linearization on the AST of the source code to extract structure- and type-aware token sequences. These sequences are transformed into a perturbation image by retrieving and reshaping each token's embedding from a learnable universal perturbation dictionary. This enables a pre-trained model originally designed for image classification to be repurposed to support code vulnerability detection, with a dynamic label remapping scheme applied at the end that reassigns the model's output to the binary vulnerability detection result. Our experiments demonstrate that Capture achieves detection accuracy comparable to state-of-the-art methods, while enhancing training efficiency due to its minimal quantity of parameters to update during the model training. Notably, Capture excels in scenarios with limited vulnerable samples, delivering superior detection accuracy and F1 scores compared to baseline methods.

1. Introduction

The fundamental origin of network attacks can often be traced back to software vulnerabilities. Despite developers' relentless efforts to enhance their code quality, the challenge of security vulnerabilities remains persistent and severe. As of 2024, the quantity of vulnerabilities publicly disclosed on the CVE website (Anon, 2024a) has exceeded 240,800, representing a nearly 40-fold increase compared to a decade ago. Recognizing the inevitability of vulnerabilities and their substantial risks, both academia and industry have been committed to devising effective detection methods (Anon, 2024b; Lipp et al., 2022; Perl et al., 2015; Kang et al., 2022). With the advent of advanced deep learning (DL) architectures and their successful applications across diverse domains, various data-driven approaches (Li et al., 2018; Chakraborty et al., 2021; Wu et al., 2022; Steenhoek et al., 2023; Tian et al., 2024) have been developed to automatically and effectively identify vulnerabilities in source code.

For vulnerability detection methods following the deep learning (DL) paradigm, deep neural network based classifiers are trained using annotated vulnerable and non-vulnerable code samples to learn features or patterns specific to vulnerabilities. These learned features are then used to uncover potential vulnerabilities within software projects. For instance, SySeVR (Li et al., 2021a) utilizes a BiGRU-based encoder to capture vulnerability-indicative features from code snippets sliced on relevant code elements. Devign (Zhou et al., 2019) operates on a hybrid graph structure that integrates the control flow graph (CFG), abstract syntax tree (AST), and data flow graph (DFG), and leverages a gated Graph Neural Network (GGNN) to extract vulnerable features. VulCNN (Wu et al., 2022), rather, converts the code snippet into an image using centrality measures on the program dependency graph (PDG) and then trains a CNN-based model for vulnerability detection.

☆ Editor: Hongyu Zhang.

* Corresponding author at: School of Computer Science and Technology, Xi'an University of Posts and Telecommunications, Xi'an, Shaanxi 710121, China.
E-mail addresses: tianzhenzhou@xupt.edu.cn (Z. Tian), sangui@stu.xupt.edu.cn (R. Qiu), ydteng@stu.xupt.edu.cn (Y. Teng), sunjiaze@xupt.edu.cn (J. Sun), chenyp@xupt.edu.cn (Y. Chen), lingwei.chen@wright.edu (L. Chen).

https://doi.org/10.1016/j.jss.2025.112365
Received 8 October 2024; Received in revised form 19 December 2024; Accepted 2 February 2025
Available online 10 February 2025
0164-1212/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Despite the gradual improvement in detection accuracy, which is a crucial and primary goal pursued by existing DL-based vulnerability detection approaches, several important aspects for practical vulnerability detection remain overlooked. (1) **Efficacy under limited data.** Compared to the abundance of benign code, real-world vulnerable samples are significantly limited (Nong et al., 2023, 2024b, 2022), due to the inadequate number of publicly disclosed vulnerabilities, which are further scattered across various vulnerability types and programming languages, as well as the high cost and rigorous domain knowledge required for manually labeling vulnerable samples. The scarcity of high-quality vulnerable samples poses a challenge for DL-based vulnerability detectors, which generally require large and diverse datasets to learn a high performing model, thereby significantly impeding their practical effectiveness. (2) **Training cost.** Current DL-based methods typically involve training models from scratch (Wu et al., 2022; Li et al., 2021a; Tian et al., 2024) by feeding the labeled samples into a DNN with randomly initialized parameters. This requires updating a large amount of parameters during training iterations, imposing significant demands on computing resources and power. Moreover, the scarcity of vulnerable samples can hinder the adequate optimization of these parameters, potentially leading to longer training time or suboptimal models that fail to capture subtle yet critical vulnerability-indicative features.

The "pre-training + fine-tuning" paradigm offers a promising solution to mitigate the learning costs by avoiding the need to always train models from scratch. For instance, CodeBERT (Feng et al., 2020), a pre-trained general-purpose programming language model, can be fine-tuned for vulnerability detection by feeding vulnerable and non-vulnerable samples. However, the fine-tuning process can still be cumbersome in practice, as it generally requires extensive updates to the pre-trained model's large amounts of parameters. This on one hand still incurs substantial computational overhead, while on the other hand poses significant challenges in achieving optimal fine-tuning, especially when the vulnerable samples are limited.

To address these challenges, we explore a new strategy that reprograms pre-trained models using simple and computationally tractable input perturbations, allowing for efficient model training and effective vulnerability detection even with limited vulnerable samples. This is inspired by observations from research on adversarial attacks (Liang et al., 2022; Zhang et al., 2023a; Goodfellow et al., 2014), where adversaries can add small, carefully crafted perturbations into the input to deliberately cause the target models to produce highly deterministic misclassifications. In this sense, by learning and appending a set of well-formulated universal perturbations to the input, a DL model trained in a source domain can be effectively repurposed to perform a task in a different target domain (Elsayed et al., 2018; Neekhara et al., 2022; Chen et al., 2022) without extensive model fine-tuning. Since only the perturbations – typically much fewer in number than the model's parameters – are trainable, this reprogramming method is significantly more efficient than conventional fine-tuning strategy and dramatically reduces the amount of training data required, while better leveraging the pre-trained model's learning capability than training from scratch.

Building on the aforementioned reprogramming framework, we present CAPTURE, a novel and highly effective **C**ross-modal **A**dversarial re**P**rogramming approach **T**owards cost-efficient v**U**lne**R**ability d**E**tection. This method repurposes high-performing computer vision models to handle semantic images transformed from the code, enabling both efficient and effective vulnerability prediction. Specifically, taking in the code snippet to analyze, CAPTURE first converts it into two sequences of normalized code tokens: one through straightforward lexical parsing and the other via structural-aware linearization of the AST to highlight the type information of code elements within the snippet. These token sequences are then transformed into a perturbation image by retrieving and reshaping each token's embedding from a learnable universal perturbation dictionary. This perturbation image is then superimposed onto a real-world image, effectively reprogramming or modulating the pre-trained computer vision model – originally designed for image classification – to now support code vulnerability detection, with additional low-cost label remapping applied at the end of the process. Our major contributions are summarized as following:

- A novel and cost-efficient vulnerability detection method CAPTURE is devised, which adopts a new paradigm by adversarially reprogramming pre-trained models from other domains to purposely address the often-overlooked data scarcity issue in real-world vulnerable samples and the high costs associated with model training.
- CAPTURE features three core designs to ensure its effectiveness: (1) light-weight generation of informative token sequences by directly parsing normalized code and linearizing the AST to capture code elements' type information; (2) ingenious perturbation formulation strategy that efficiently reshapes and organizes token embeddings sourced from a perturbation dictionary to generate perturbation images to facilitate modulating the pre-trained Vision Transformer (ViT) model (Dosovitskiy et al., 2021); (3) dynamic label remapping scheme, where label mappings between the original and target tasks are not fixed from the outset but are dynamically adjusted according to the updates of the perturbations during training, better mapping the original task's output to vulnerability detection outcomes.
- Extensive experiments are conducted on multiple real-world datasets to evaluate CAPTURE's vulnerability detection capability. The results indicate that CAPTURE not only matches SOTA methods in detection accuracy, but also improves training efficiency as a result of the considerably fewer parameters that need to be fine-tuned. Notably, CAPTURE exhibits superior performance particularly when the vulnerable sample quantity is limited. Additionally, our ablation studies have validated the effectiveness of the dynamic label remapping scheme over other strategies and the active role of the hybrid sequence representation. The source code implementation of CAPTURE is publicly available at https://github.com/XUPT-SSS/CAPTURE to facilitate further research.

The rest of this paper is organized as follows. Section 2 discusses the preliminaries on adversarial attacks and reprogramming. Section 3 introduces the overall framework of CAPTURE and its core designs. Section 4 presents the experimental settings and evaluation results. Section 5 discusses the limitations and future work. The related works are reviewed in Section 6, and finally Section 7 concludes.

2. Preliminary

This section offers an essential overview of adversarial attack and adversarial reprogramming to clarify the motivation behind our devised model CAPTURE.

2.1. Adversarial attacks

Neural networks, with their remarkable learning capabilities, autonomously produce results through their architectures and algorithms, relying heavily on extensive external data during training. However, this reliance makes them vulnerable to adversarial examples—inputs subtly modified in ways that can deceive the model (Goodfellow et al., 2014). Adversarial attacks exploit this sensitivity by introducing imperceptible perturbations to input data, causing the model to generate incorrect outputs or misclassify examples, thereby threatening the security of deep learning in practical applications.

These attacks can are typically categorized into targeted and non-targeted attack, where targeted attacks cause the model to misclassify the input into a specific incorrect class and nontargeted attacks expect adversarial examples to be misclassified as any incorrect class (Yuan et al., 2019). Different adversarial attacks have been accordingly proposed, such as Fast Gradient Sign Method (FGSM) (Goodfellow et al., 2014), Projected Gradient Descent algorithm (PGD) (Madry, 2017),

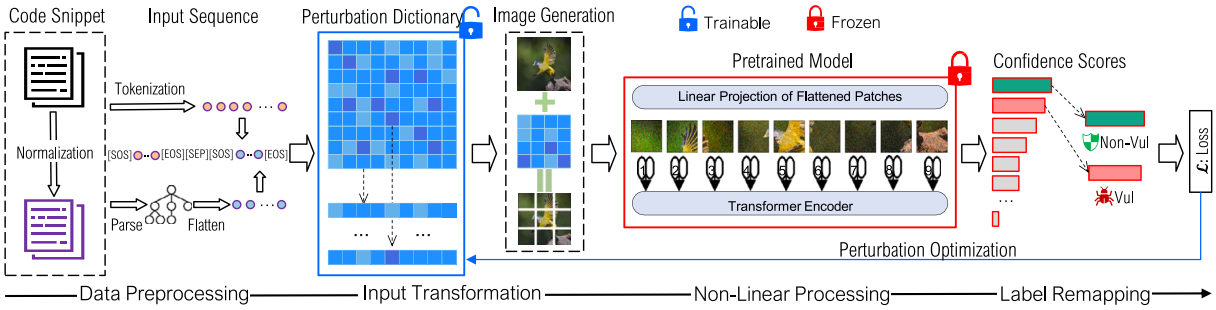

Fig. 1. The overview of CAPTURE.

and DeepFool (Moosavi-Dezfooli et al., 2016), which utilize gradients in feature space (*e.g.*, images) to generate perturbations. In general, perturbations are differently applied to different inputs to induce misclassification, while recently, some efforts have focuses on crafting universal perturbations, which can be added to many different inputs to generate adversarial examples (Moosavi-Dezfooli et al., 2017; Baluja and Fischer, 2018). In this work, we aim to learn such a universal perturbation to implement adversarial reprogramming.

2.2. Adversarial reprogramming

Adversarial reprogramming (Chen, 2024) differs from adversarial attacks in its objective, operation, and constraint: (1) instead of degrading model performance or causing misclassifications, it seeks to reprogram a model pre-trained in a source domain to perform tasks that can be rather different in a target domain (Elsayed et al., 2018), without directly calculating the expected output for each input; (2) this reprogramming is achieved by learning a universal perturbation applied to the input, while keeping the pre-trained model architecture and parameters remain unchanged during training (Chen et al., 2022); and (3) the universal perturbation is learned as model parameters that interact solely with inputs in the embedding space through the non-linear model structure, eliminating the need to generate adversarial examples or consider the perturbation imperceptibility and plausibility of the perturbed inputs. More specifically, given a model s pre-trained for a source-domain task, which produces outputs $s(x_s)$ for inputs $x_s \in \mathcal{X}_s$, and a target-domain task t that expects outputs $t(x_t)$ for inputs $x_t \in \mathcal{X}_t$, adversarial reprogramming is achieved by learning an input transformation function $h_s(\cdot;\theta)$ and a label remapping function $h_t(\cdot)$ that bridge the two different tasks. The function $h_s(\cdot;\theta)$ transforms a target-domain input x_t into a source-domain input $\mathbf{x}_s$ in the embedding space, while the function $h_t(\cdot)$ maps the source-domain output $s(\mathbf{x}_s)$ back to the target-domain output $t(x_t)$. These transformation functions can be defined as follows:

$$\mathbf{x}_s = h_s(x_t;\theta),\ t(x_t) = h_t(s(\mathbf{x}_s)) \tag{1}$$

where θ is the universal perturbation, which is the only trainable parameter throughout this process.

With its small number of trainable parameters, adversarial reprogramming significantly reduces the need for labeled data and computational cost, while still effectively leveraging the pre-trained model's capabilities to extract subtle semantic patterns from complex input samples. These unique advantages inspire us to extend the concept of adversarial reprogramming to the field of program analysis, enabling precise code vulnerability detection at the function level that addresses the challenges discussed in Section 1.

3. Design of CAPTURE

In this section, we provide a comprehensive explanation of the technical design of CAPTURE, the overview of which is illustrated in Fig. 1. Given a code snippet (*i.e.*, a function), CAPTURE is structured around four key modules, including (1) data preprocessing which normalizes the source code and represents it as a token sequence that encodes both code semantic and structure information; (2) input transformation that converts the extracted token sequence into a perturbation image by retrieving and reshaping each token's embedding from a learnable universal perturbation dictionary; (3) non-linear processing that leverages the self-attention within the pre-trained ViT model to translate perturbations into model parameters and adjust the classification outputs; (4) label remapping that maps ViT's original image classification outputs to the corresponding vulnerability prediction outcomes. Throughout this process, the universal perturbation dictionary is trained, and the finalized CAPTURE model accepts raw code functions as inputs and predict their vulnerabilities.

3.1. Data preprocessing

3.1.1. Code normalization

By normalizing variable and function names, we reduce the complexity of the token space and minimize the risk of the code semantic learning relying on superficial cues, such as specific naming conventions, which may vary widely across developers, or embed strong indications to mislead vulnerability detection. This process also mitigates the impact of out-of-vocabulary tokens, such as user-defined identifiers, which can otherwise introduce noise and hinder the model's generalization capabilities. Furthermore, the removal of comments eliminates extraneous information that, while beneficial for human readability, does not contribute to the execution or vulnerability of the program. In this respect, to focus on the structural and functional aspects of the code, CAPTURE first removes the code comments, and then substitutes each variable and the name of each user-defined function with "`VARi`" and "`FUNi`", where i corresponds to the order of their first appearance in the code. A normalization processing example is given in Fig. 2.

3.1.2. Code representation

Selecting an appropriate representation of code is critical for accurately capturing its semantic and structural characteristics. Early methods treated source code as a linear sequence of lexically parsed code tokens, relying on neural networks such as CNN (Li et al., 2017) or RNN (Xu et al., 2018) to learn code representations. While these approaches worked for simpler representations, they fell short of fully capturing the complex structural relationships inherent in code. With the emergence of Transformer architectures (Vaswani et al., 2017), sequence-based models became more popular, given their superior capabilities for processing sequential data. However, unlike natural language, code features rich and intricate structural information that is crucial for understanding its functionality at a finer granularity. To address this, we incorporate both token sequences and ASTs into our model's input, ensuring that both semantic and structural aspects of the given code snippets are preserved.

Specifically, the input to the model is constructed from two components: token sequences and ASTs, allowing for a nuanced representation of the code. Given a code snippet $C \in \mathcal{C}$, we first tokenize it using

Fig. 2. An normalization processing example.

a lexer, which produces a sequence of tokens $C = \{C_1, C_2, \ldots, C_m\}$, capturing the basic programming logic and semantics of the code. This token sequence represents the basic building blocks of the code. However, token sequences alone cannot represent the structural information embedded within the code's organization. To capture this structural information, we employ ASTs, which have been extensively used in diverse code analysis tasks (Lin et al., 2017; Qian et al., 2023; Hu et al., 2023), and represent the hierarchical structure of the code, providing a more detailed view that token sequences cannot express. Using the tree-sitter[1] to construct the AST from C, we further parse it into a sequence via a depth-first traversal. An AST typically contains both lexical and syntactic information, with lexical data residing primarily in its terminal nodes, which are already represented by the token sequence. Therefore, during AST traversal, we retain only non-terminal nodes, yielding the sequence $A = \{a_1, a_2, \ldots, a_k\}$ that encodes the structural hierarchy of the code, such as function definitions, declarations, and parameter lists. Maintaining the sequence order is crucial, as any reordering could disrupt the code's intended functionality. More importantly, the connections between these elements provide essential contextual information for comprehending the structure of the code.

To prepare the final input, we concatenate the token and AST sequences with special tokens marking the boundaries of each sequence. Specifically, we add **[SOS]** at the start and **[EOS]** at the end of each sequence to indicate their beginning and end, and insert **[SEP]** to separate the two sequences. The resulting input representation is formally expressed as:

$$X_t = \{[SOS], C_1, \ldots, C_m, [EOS], [SEP], [SOS], a_1, \ldots, a_k, [EOS]\} \tag{2}$$

Fig. 3 illustrates this process with a simple example. This dual representation ensures that the model can leverage both the semantic logic captured by the token sequence and the structural relationships represented by the AST. As a result, the model gains a more comprehensive understanding of the code, leading to improved vulnerability detection performance.

[1] https://treesitter.github.io/tree-sitter/

3.2. Input transformation

We opt to reprogram ViT (Dosovitskiy et al., 2021) for vulnerability detection due to its strong capacity of capturing long-range dependencies through self-attention, making it particularly effective at understanding the intricate relationships between code tokens and their structural representations. This global context modeling aligns with function-level vulnerability detection, where both semantic and structural nuances are critical, in contrast to traditional CNNs (Iandola et al., 2014; He et al., 2016; Szegedy et al., 2016), which are more focused on local patterns. As discussed in Section 3.1, our processed inputs are sequences that are different from images used by ViT. To bridge this gap, we introduce an input transformation function $h_s(\cdot;\boldsymbol{\theta})$ that converts code sequences into a representation compatible with image classifiers by incorporating trainable universal perturbations, such that the pre-trained model can be adapted to generate the expected outputs.

3.2.1. Formulating perturbation dictionary

To transform a sequence of discrete tokens into a perturbation image, existing sequence-based adversarial reprogramming techniques (Hambardzumyan et al., 2021; Neekhara et al., 2018) typically first learn token embeddings, which are then integrated with the additional perturbations to generate the final input representation. However, such an input transformation design may significantly increase the number of trainable parameters throughout the adversarial reprogramming process, leading to higher computational costs and a greater need for labeled data—both of which contradict our goal for efficient training and effective vulnerability detection with limited samples. In practice, token embeddings are often implemented as a simple lookup table that stores embeddings of a fixed sized dictionary, where each input sequence retrieves its token embeddings using the corresponding indices. This offers some key advantages: this lookup table is trainable, universally applicable to all input token sequences within the code corpus, and can modulate the way the input is processed by the pre-trained model—essentially functioning as universal perturbations. In this sense, we design an embedding lookup table, referred to as the perturbation dictionary throughout the paper, which directly serves as universal perturbations to manipulate input sequences instead of adding extra perturbations, keeping model parameters and computational overhead low. Formally, let $\mathcal{X}_t$ denote the set of input sequences from Eq. (2), and let n be the number of unique code tokens in $\mathcal{X}_t$. The perturbation dictionary is formulated as a matrix $\boldsymbol{\Theta} \in \mathbb{R}^{n\times d}$, where d is the embedding dimension. $\boldsymbol{\Theta}$ is initialized with random values and optimized during training.

3.2.2. Constructing perturbation images

By looking up the perturbation dictionary $\boldsymbol{\Theta}$, an input sequence $X_t \in \mathcal{X}_t$ can be mapped into an embedding sequence $\mathbf{X}_t$. Without loss of generalizability, we represent this sequence as $\mathbf{X}_t = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_l]$, where l specifies the sequence length, and each token embedding vector $\mathbf{x}_i$ is then reshaped into a patch of size $p \times p \times 3$. Therefore, the embedding dimension d must satisfy $d = p\times p\times 3$ to ensure compatibility with the patch size. Consider an image for the ViT model, where $\mathbf{X}_s \in \mathbb{R}^{h\times w\times 3}$ with h and w representing the image height and width, respectively, and initialized as a zero matrix. The perturbation image can be accordingly constructed by sequentially arranging patches from the embedding sequence, starting from the top-left corner and moving to the bottom-right, as described in Algorithm 1.

The perturbation dictionary $\boldsymbol{\Theta}$ serves as the only learnable parameter tensor throughout the adversarial reprogramming process. The number of parameters is determined by $n \times d$. The length of the input sequence that can be accommodated in the image depends on the patch size p and the image height h and width w. To standardize the input length, we impose a fixed sequence length of l: for sequences shorter than l, we apply padding using special tokens; for sequences longer

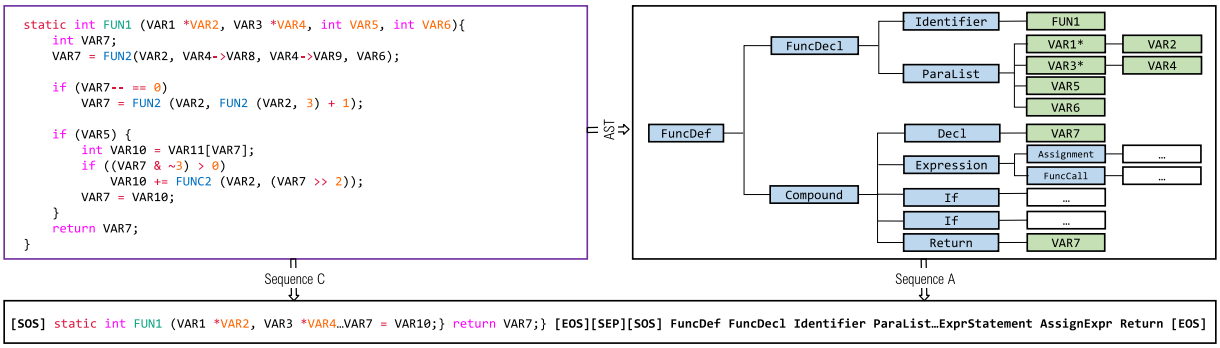

Fig. 3. An example of representing normalized code snippet by concatenating the token and AST sequences.

than l, we truncate the excess tokens. Some previous work on adversarial reprogramming (Chen et al., 2022; Elsayed et al., 2018; Chen et al., 2021) has explored embedding the transformed input $\mathbf{X}_s$ within real-world images to conceal its visibility, demonstrating improvements in the convergence speed of model training. While hiding universal perturbations is not necessary for our task, we still embed $\mathbf{X}_s$ into a real-world image to expedite the training process.

Algorithm 1 Input transformation $h_s(X_t; \boldsymbol{\Theta})$

Input: $X_t = \{x_1, x_2, \ldots, x_l\}$ and $\boldsymbol{\Theta} \in \mathbb{R}^{n \times d}$
Output: $\mathbf{X}_s \in \mathbb{R}^{h \times w \times 3}$
Generate $\mathbf{X}_t = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_l\}$ by looking up $\boldsymbol{\Theta}$
$\mathbf{X}_s \leftarrow \mathbf{0}$
for $\mathbf{x}_i \in \mathbf{X}_t$ **do**
  $\mathbf{x}_i \in \mathbb{R}^{p \times p \times 3} \leftarrow \mathbf{x}_i \in \mathbb{R}^d$
  $r \leftarrow \lfloor ((i-1) \times p)/h \rfloor$
  $c \leftarrow ((i-1) \times p) \mod w$
  $\mathbf{X}_s[r : r+p, c : c+p, :] \leftarrow \mathbf{x}_i$
end for
Return $\mathbf{X}_s$

3.3. Non-linear processing

Most prior adversarial reprogramming approaches, which relies on additive perturbations to input embeddings (Tsai et al., 2020; Chen et al., 2022; Elsayed et al., 2018), require non-linear interactions between them to translate these perturbations into effective model parameters. In contrast, CAPTURE employs trainable token embeddings as perturbations, allowing them to directly function as model parameters that adapt the pre-trained model more efficiently. This streamlines the process, while non-linear self-attention can be relaxed to focus on learning intricate feature patterns from perturbation images, effectively capturing vulnerability-specific semantics. Specifically, CAPTURE proceeds by feeding the perturbation image $\mathbf{X}_s$ into the pre-trained ViT, in which the perturbation image is divided into fixed-size patches, with each of them being projected into a vector, resulting in a sequence $\mathbf{X}_s = \{\mathbf{x}_{s1}, \mathbf{x}_{s2}, \ldots, \mathbf{x}_{sn}\}$. This vector sequence is then passed through the encoder featuring multi-layer transformers to perform multi-head self-attention operations. Each attention head is calculated as follows:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\mathbf{T}}}{\sqrt{d_k}}\right)\mathbf{V} \tag{3}$$

$$\mathbf{Q} = \mathbf{W}^Q\mathbf{X}_s, \mathbf{K} \quad = \mathbf{W}^K\mathbf{X}_s, \mathbf{V} = \mathbf{W}^V\mathbf{X}_s \tag{4}$$

Here, Attention$(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \in \mathbb{R}^{n \times d_v}$ denotes the matrix of attention scores, with each row being a d_v-dimensional vector that corresponds to a specific input patch. The matrices $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$, $\mathbf{K} \in \mathbb{R}^{n \times d_k}$, and $\mathbf{V} \in \mathbb{R}^{n \times d_v}$ are computed by multiplying $\mathbf{X}_s$ by the pre-trained weight matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, and $\mathbf{W}^V$ of the ViT model. The term $\frac{1}{\sqrt{d_k}}$ acts as a scaling factor to ensure that the dot products do not become too large, which would result in overly sharp softmax distributions, destabilizing training. This enables the model concentrating more on the pertinent patches of the perturbation image $\mathbf{X}_s$, effectively highlighting areas of the code representation likely to harbor vulnerabilities, thus enhancing detection performance.

3.4. Label remapping

Once the ViT processes the input perturbation image $\mathbf{X}_s$, it outputs a prediction vector:

$$\mathbf{z}_s = \text{ViT}(\mathbf{X}_s) \tag{5}$$

with a dimensionality of 1,000, corresponding to the model's pre-trained image classification task. However, since our goal is binary vulnerability detection, it is essential to map this output to a 2-dimensional space. This is achieved through a label remapping function, $h_t(\mathbf{z}_s)$, which transforms the high-dimensional prediction $\mathbf{z}_s$ into binary predictions tailored to our specific task. To implement label remapping, most existing work defines a one-to-one correspondence between the pre-trained model's prediction and the target class space. These methods generally fall into two categories (Chen et al., 2023):

- Random label mapping (RLM), which randomly selects source labels to the target domain without using or even considering any prior knowledge or information from the source model (Elsayed et al., 2018; Chen et al., 2022).
- Confidence-based label mapping (CLM), which aligns target labels with source labels according to the prediction confidence scores of the source model (Tsai et al., 2020; Neekhara et al., 2022).

While both label mapping approaches have demonstrated their effectiveness in facilitating adversarial reprogramming, they are static, using a one-time mapping. In other words, Once source labels are initially matched to target labels, these mappings remain fixed throughout training, failing to account for the dynamic updates to the model's parameters (*i.e.*, perturbation dictionary), which, in turn, impairs the model's ability to fully optimize the input perturbations and adapt to evolving patterns during the training process.

Inspired by the dual-layer optimization strategy, where the upstream task adjusts its strategy based on the downstream task's response, we design a dynamic label mapping (DLM) scheme to address the aforementioned limitation. Instead of treating label mapping as an isolated function, we incorporate it into a comprehensive end-to-end learning framework. In this setup, the optimization of the perturbation dictionary $\boldsymbol{\Theta}$ introduced in the input transformation serves as the upstream task, and the label remapping functions as the downstream task, where the updated prediction confidence scores produced by the ViT dynamically decide the source labels to be mapped to the vulnerability detection results. Specifically, the label remapping function $h_t(\mathbf{z}_s)$ ranks the confidence scores in $\mathbf{z}_s$, and selects the top two highest probabilities to predict the absence and presence of vulnerability, respectively,

which is formulated as:

$$p_t = h_t(\mathbf{z}_s) = \begin{cases} \max_{i \in \mathcal{Y}} (\mathbf{z}_s)_i & y_t = 0 \\ \max_{j \in \mathcal{Y}, j \neq i} (\mathbf{z}_s)_j & y_t = 1 \end{cases} \tag{6}$$

The optimization of the perturbation dictionary $\boldsymbol{\Theta}$ is to minimize the cross-entropy loss between the true vulnerability labels y_t and the model's predicted scores p_t derived from $h_t(\mathbf{z}_s)$ in Eq. (6), which is specified as:

$$\boldsymbol{\Theta} = \underset{\boldsymbol{\Theta}}{\operatorname{argmin}}(- \sum_{X_t \in \mathcal{X}_t} y_t \log p_t + \lambda \|\boldsymbol{\Theta}\|_F^2), \tag{7}$$

where λ denotes the regularization parameter. At each training iteration, the output of $h_t(\mathbf{z}_s)$ is used to facilitate optimization, while the updated perturbation dictionary is leveraged to compute new prediction scores $\mathbf{z}_s$, subsequently assigning updated probabilities for performing label mapping. As the dual-layer optimization progresses, the convergence of $\boldsymbol{\Theta}$ drives the alignment and convergence of label mapping. The specific implementation details are outlined in Algorithm 2.

Algorithm 2 Label remapping $h_t(\mathbf{z}_s)$ and optimization

Input: Input sequences $\mathcal{X}_t$, training epochs T.
Output: Perturbation dictionary $\boldsymbol{\Theta}$
for training epoch $\leq T$ **do**
  for $X_t \in \mathcal{X}_t$ **do**
    $\mathbf{X}_s \leftarrow h_s(X_t; \boldsymbol{\Theta})$
    $\mathbf{z}_s \leftarrow \text{ViT}(\mathbf{X}_s)$
    $p_t \leftarrow h_t(\mathbf{z}_s)$
  end for
  $\boldsymbol{\Theta} \leftarrow \operatorname{argmin}(- \sum_{X_t \in \mathcal{X}_t} y_t \log p_t + \lambda \|\boldsymbol{\Theta}\|_F^2)$
end for

4. Experiments and evaluations

Extensive experiments are conducted in this section to assess the performance of Capture, with the goal of addressing the following four questions:

- **RQ1: Vulnerability detection performance.** How does Capture perform compared to other DL-based vulnerability detectors?
- **RQ2: Performance under data-limited scenarios.** How do Capture and the other vulnerability detection models perform with limited training samples?
- **RQ3: Impact of pre-trained model.** How does the backbone pre-trained model impact Capture's detection performance?
- **RQ4: Ablation Study.** How do the label remapping strategy and our mixed token sequence representation contribute to Capture's detection performance?
- **RQ5: Efficiency analysis.** How about the runtime overhead of Capture compared against the other models?

4.1. Experimental setup

4.1.1. Datasets

The D2 A and Devign datasets, which are widely used in the literature of DL-based vulnerability prediction, are used for the evaluation and comparison of Capture with other baseline models. The D2A (Zheng et al., 2021) dataset is a real-world vulnerability dataset constructed by the IBM from popular open-source projects like OpenSSL, httpd, Nginx, and LibTiFF. The Devign (Zhou et al., 2019) dataset rather comprises real-world C/C++ functions from QEMU and FFmpeg, created by collecting vulnerable functions from vulnerability-related commits, followed by manual verification by security analysts. The Big-Vul (Fan et al., 2020) is constructed by crawling the CVE entries and related code repositories, encompassing 348 distinct projects and 91 vulnerability types. However, there is a significant imbalance between vulnerable and non-vulnerable samples in the dataset. A brief overview of these datasets is provided in Table 1.

Table 1
Basic information of the vulnerability datasets.

Dataset	#Vul	#NonVul	Vul_Ratio (%)	Dataset Type
D2A	2795	2444	53.35	Real-World
Devign	12,460	14,858	45.61	Real-World
BigVul	10,900	177,736	5.78	Real-World

4.1.2. Baseline models

We compare Capture against seven other DL-based vulnerability detectors, including two widely-used base models: TextCNN and BiLSTM, as well as five advanced methods: SySeVR, TrVD, VulCNN, Reveal, and EPVD, and three fine-tuned pre-trained models: CodeBERT, UniXCoder and GraphCodeBERT. A brief introduction to these methods is as below.

- **SySeVR** (Li et al., 2021a) performs code slicing on the target function's Program Dependency Graph (PDG) according to the supposed vulnerability-informative hotpots such as certain API calls and arithmetic expressions. The token sequence of the sliced code is then input into a Bidirectional Gated Recurrent Units (BiGRU) for feature distillation and vulnerability prediction.
- **VulCNN** (Wu et al., 2022) processes the function's PDG nodes with the Sent2Vec (Pagliardini et al., 2018) to generate node vectors, and calculates degree, Katz, and closeness centralities as importance weights for each node. These weighted node vectors are then regarded as image channels, while a CNN is used to extract features for vulnerability prediction.
- **TrVD** (Tian et al., 2024) is a tree-based detection model, which utilizes an attention-augmented tree-structured RNN to collect features from the segmented sub-trees of the AST, and then distills indicative ones with a Transformer-based aggregator for vulnerability prediction.
- **Reveal** (Chakraborty et al., 2021) is a graph-based vulnerability detector that employs the Graph Gated Neural Networks (GGNN) to distill features of vulnerability-significance from the Code Property Graph (CPG) for predictive analysis.
- **EPVD** (Zhang et al., 2023b) builds syntax-CFG and decomposes it into multiple paths, from which features are extracted using a combination of CodeBERT and a CNN, applying intra- and inter-path attention. The resulting feature vectors are then passed into an MLP for vulnerability detection.
- **CodeBert** (Feng et al., 2020), **UniXCoder** (Guo et al., 2022) and **GraphCodeBERT** (Guo et al., 2021) are three widely used pre-trained code models, where we fine-tune them on the vulnerability datasets for vulnerability prediction in this work.

4.1.3. Implementation and parameter settings

Capture is implemented in python, utilizing the tree-sitter library for code parsing and the PyTorch framework for setting up the adversarial reprogramming architecture and training the model. ViT-Base (Dosovitskiy et al., 2021) is utilized as Capture's default backbone, which is pre-trained on plentiful images with dimensions of $384 \times 384 \times 3$ and divided into patches with dimensions of $16 \times 16 \times 3$. Accordingly, perturbation images generated by the input transformation module are also fixed at this size, allowing each image to encode up to 576 code tokens.

For experimental evaluation settings, both datasets are randomly shuffled and split with an 8:1:1 ratio for training, validation, and testing. The learning rate is initialized at 1e-2, using the Adam optimizer with a batch size of 16. A cosine annealing algorithm with a half-cycle of 10 is applied for learning rate decay to avoid local optima. After each epoch, validation accuracy is computed, and if there is no improvement for five consecutive epochs, early termination occurs. All experiments are carried out on two Linux servers running Ubuntu 20.04, each equipped with identical hardware configurations of two Intel Silver CPUs, 128 GB of RAM, and an RTX3090 GPU.

4.1.4. Evaluation metrics

In line with the performance metrics widely adopted in the DL-based vulnerability detection literature, Accuracy, Precision, Recall, and F1 are used for a fair comparison with the baselines. These specific metrics are formulated as follows:

$$Acc = \frac{TP + TN}{TP + FP + TN + FN} \tag{8}$$

$$Prec = \frac{TP}{TP + FP}, \quad Rec = \frac{TP}{TP + FN} \tag{9}$$

$$F1 = 2 \times \frac{Prec \times Rec}{Prec + Rec} \tag{10}$$

where TP represents the number of correctly detected vulnerable samples, FP is the number of incorrectly classified benign samples, TN refers to the number of correctly predicted benign samples, and FN refers to the number of vulnerable samples that are failed to be detected by the model.

4.2. Experimental results

4.2.1. RQ1: Vulnerability detection performance

In this experiment, we evaluate the vulnerability detection capability of CAPTURE against the other baseline detectors. As shown in Table 2, CAPTURE achieves 63.93% accuracy and 62.08% F1 score on the D2 A dataset, surpassing all the baselines with an average relative improvement of 8.4% and 9.7%, respectively. Similar results are observed on the Devign dataset, where CAPTURE outperforms the other methods with averaged relative increase of 5.3% and 14.7% on accuracy and F1 score. On the Big-Vul dataset, CAPTURE also outperforms other methods with an average relative accuracy improvement of 4.5% and a striking 108.1% increase in F1 score. It is worth noting that while all methods maintain consistently high accuracy, their F1 scores remain significantly lower. This discrepancy highlights the impact of the extreme imbalance between vulnerable and non-vulnerable samples, which skews DL-based detectors towards classifying functions as non-vulnerable. Further analysis of precision and recall reveals that relatively simpler models, such as BiLSTM and TextCNN, tend to generate more false positives, while more complex models based on Transformer architectures, like TrVD, EPVD, and fine-tuned PLMs, produce more false negatives. Notably, EPVD, which incorporates advanced CFG path generation along with a combination of CodeBERT and CNN for intra- and inter-path feature extraction, achieves the highest precision at 70.59%, but records the lowest recall at 2.22%. The model's intricate design places significant emphasis on the fine details of vulnerable patterns, resulting in an overly cautious approach to determining vulnerability. As a consequence, while it effectively reduces false positives, it also fails to identify many functions that are actually vulnerable. Overall, the DL-based vulnerability prediction methods still have a long way to go. The less-than-optimal detection performance suggests that the intricate and diverse coding patterns in real-world code present significant challenges for vulnerability detection.

Answer to RQ1: Among the evaluated DL-based vulnerability detectors, CAPTURE demonstrates superior performance in terms of accuracy and F1. However, the reliable detection of vulnerabilities implicit in real-world programs remains a challenge, necessitating continued advancements in DL-based methods to better comprehend the intricate vulnerability patterns within the diverse code context.

4.2.2. RQ2: Performance under data-limited scenarios

While the DL-based models have shown promising vulnerability detection performance compared to the conventional rule-based methods (Wu et al., 2022), their effectiveness heavily depends on training the feature encoding model with rich number of labeled samples (Nong et al., 2024b). However, obtaining an adequate amount of labeled vulnerable code in real world is actually highly challenging (Nong et al., 2024c), especially for specific vulnerability types where the available instances are often scarce. In this experiment, we explore the performance of CAPTURE and baseline models when faced with limited training data. To simulate these conditions, 10, 50, 100, 500, and 1,000 instances are randomly picked from the training samples of the three datasets, meanwhile ensuring that the validation sets and the test sets remain unaltered. We repeat this process for 10 times, with the metric values obtained on the test sets averaged as the results for analysis.

Table 2
Performance Comparison with the Baselines.

Dataset	Detector	Acc.	F1.	Prec.	Rec.
D2A	BiLSTM	54.19	55.02	55.85	54.22
	TextCNN	55.37	52.84	58.20	48.38
	SySeVR	60.56	57.55	66.67	50.63
	Reveal	58.58	53.58	64.53	45.80
	VulCNN	58.54	58.14	58.22	**58.07**
	TrVD	59.06	57.93	61.76	54.55
	EPVD	61.74	55.81	**69.23**	46.75
	CodeBERT	62.10	56.76	69.01	48.20
	UniXCoder	60.91	59.55	63.91	55.74
	GraphCodeBert	59.56	59.42	61.62	57.38
	CAPTURE	**63.93**	**62.08**	67.95	57.14
Devign	BiLSTM	53.26	41.72	45.52	38.50
	TextCNN	54.25	45.27	47.13	43.56
	SySeVR	51.84	49.72	**56.66**	44.29
	Reveal	54.37	43.48	47.43	40.14
	VulCNN	56.17	48.87	52.49	45.71
	TrVD	56.19	49.13	52.64	46.06
	EPVD	58.52	53.40	53.56	53.24
	CodeBERT	57.96	45.49	50.05	41.68
	UniXCoder	57.73	48.08	49.76	46.51
	GraphCodeBert	57.21	44.02	48.96	39.98
	CAPTURE	**58.57**	**53.53**	52.20	**54.93**
Big-Vul	BiLSTM	88.39	25.81	20.68	**34.32**
	TextCNN	90.17	26.86	23.88	30.68
	SySeVR	90.10	19.34	30.91	14.08
	Reveal	87.14	22.87	17.22	**34.04**
	VulCNN	80.60	26.38	55.85	17.27
	TrVD	94.37	10.65	58.46	5.86
	EPVD	94.34	4.30	**70.59**	2.22
	CodeBERT	94.48	22.19	55.56	13.86
	UniXCoder	94.42	24.56	55.16	15.80
	GraphCodeBert	94.58	23.75	61.87	14.70
	CAPTURE	**94.74**	**31.36**	62.86	20.89

As depicted in Fig. 4, CAPTURE consistently outperforms the baselines across nearly all the evaluated data-limited training settings with real-world samples. On the D2 A dataset, CAPTURE illustrates an average relative improvements of 6.5% and 19.5% over its counterparts in terms of the accuracy and F1 metrics, respectively. Similarly, on the Devign dataset, it shows a relative 4.7% gain in accuracy and 15.7% increase in F1; while on the Big-Vul dataset, significant relative improvements of 23.3% in accuracy and 16.9% in F1 score are achieved. Remarkably, with just 100 training samples, CAPTURE achieves 58.05% accuracy, outperforming BiLSTM, TextCNN, SyseVR, Reveal and VulCNN models trained with ten times more data on D2 A; while on the Devign and Big-Vul datasets, CAPTURE, trained with 100 samples, reaches accuracies of 55.56% and 86.17%, respectively, surpassing all baselines except for CodeBERT on the Devign dataset and TrVD on the Big-Vul dataset, both of which are trained with 1,000 samples. The exhibited performance superiority of CAPTURE can be attributed to the much less perturbation parameters to tune, with which the pre-existing knowledge in the pre-trained model can be more efficiently leveraged even with limited data during adversarial reprogramming. Additionally, it can be observed that all methods present a general upward trend in performance as the number of training samples increases. This reconfirms the importance of abundant and high-quality data in boosting DL-based models, while our adversarial programming based CAPTURE provides a promising way to mitigate this issue.

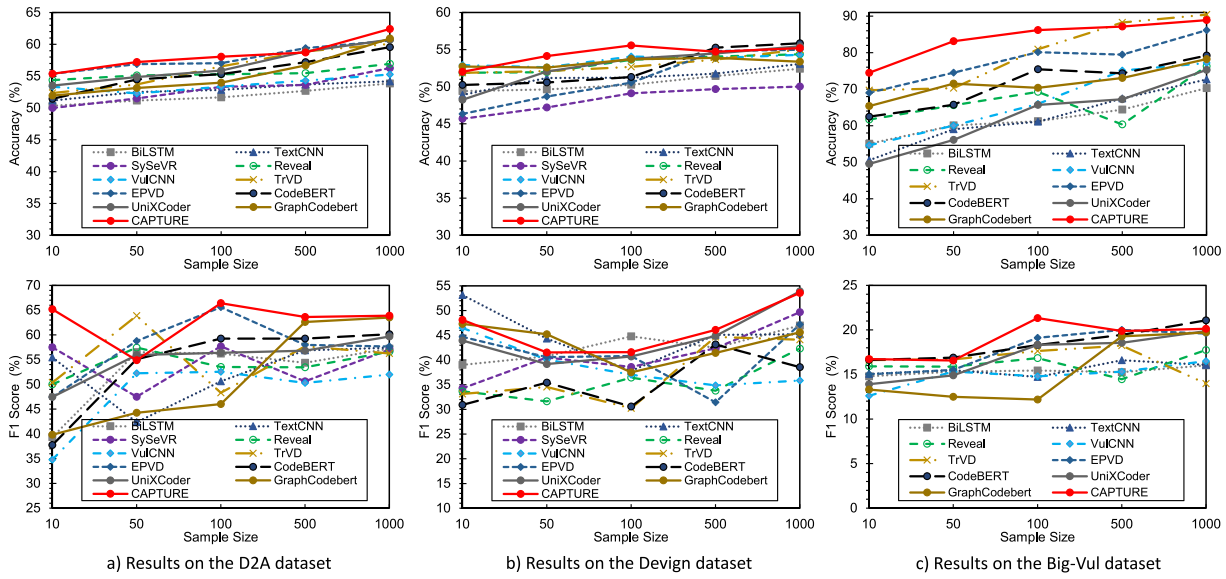

Fig. 4. Performance of the DL-based models under data-limited scenarios.

Table 3
Performance of Capture with different backbone.

Dataset	Vision Model	Metrics			
		Acc	F1	Prec	Rec
D2A	ResNet18	56.88	61.58	57.06	**66.88**
	ResNet50	60.57	60.64	62.63	58.77
	ViT	**63.93**	**62.08**	**67.95**	57.14
Devign	ResNet18	55.97	25.88	48.17	17.69
	ResNet50	57.17	43.15	50.98	37.41
	ViT	**58.57**	**53.53**	**52.20**	**54.93**
Big-Vul	ResNet18	94.05	17.24	47.37	10.54
	ResNet50	93.75	23.49	41.95	16.31
	ViT	**94.74**	**31.36**	**62.86**	**20.89**

Table 4
Performance of Capture with different label remapping strategies.

Dataset	Mapping Strategy	Metrics			
		Acc	F1	Prec	Rec
D2A	RLM	58.39	60.63	59.32	**62.01**
	CLM	61.74	62.00	63.70	60.39
	DLM	**63.93**	**62.08**	**67.95**	57.14
Devign	RLM	56.73	47.38	50.24	44.82
	CLM	58.20	48.14	**52.22**	44.65
	DLM	**58.57**	**53.53**	52.20	**54.93**
Big-Vul	RLM	94.20	20.08	52.99	12.39
	CLM	94.31	25.49	56.85	16.43
	DLM	**94.74**	**31.36**	**62.86**	**20.89**

Answer to RQ2: Capture exhibits impressive detection capability when the number of real-world samples is limited, outperforming all baselines across the evaluated data-limited scenarios, offering a promising solution to alleviate the challenge of vulnerability sample scarcity.

4.2.3. RQ3: Impact of pre-trained model

The pre-trained model serves as an important constituent part of our approach. Besides the advanced ViT that Capture adversarially reprograms on, other widely used pre-trained vision models can also be adopted. Herein, to investigate the versatility of Capture in adapting with different backbones, we evaluate its performance using two popular alternatives, ResNet-18 and ResNet-50, in comparison to the ViT-based version.

As shown in Table 3, Capture with ViT as its backbone consistently surpasses the alternative models across the majority of metrics. Specifically, replacing ViT with ResNet-18 and ResNet-50 results in accuracy drops of 7.1% and 3.4%, and F1 score decreases of 0.5% and 1.4% on the D2 A dataset, respectively. Similar declines are observed on the other two datasets: on the Devign dataset, ViT achieves the highest scores, with average decreases of 2.0% in accuracy and 19.0% in F1 score when substituted by alternative models; on the Big-Vul dataset, accuracy and F1 scores drop by an average of 0.8% and 11.0%, respectively, under the same substitutions. This indicates that the selection of backbone affects to some extent the adversarial reprogramming performance for the vulnerability detection task. Generally, models with larger parameter volumes, which offer greater feature distillation capabilities, outperform smaller models. Particularly, ViT's larger number of pre-trained parameters and its advanced self-attention mechanism, which excels at capturing long-range global-context features, make it the best performing and most adaptable backbone for Capture.

Answer to RQ3: While different pre-trained backbone models can fit into Capture, their reprogrammed performance in vulnerability detection vary to noticeable extent. The ViT, with larger parameter capacity and advanced self-attention mechanism for capturing global-context features, presents to be the most effective backbone for Capture.

4.2.4. RQ4: Ablation study

As discussed in Section 3.4, we introduce a new dynamic label remapping strategy with the expectation of further refining the model reprogramming effectiveness for enhanced vulnerability prediction capability. In this section, the performance of Capture are assessed and compared under the three different label remapping strategies (*i.e.*, random label mapping (RLM), confidence-based label mapping (CLM), and our proposed dynamic label mapping (DLM)).

As illustrated by the metric values in Table 4, Capture using the DLM label remapping strategy achieves the best overall vulnerability detection performance. Specifically, on the D2 A dataset, it delivers average relative improvements of 6.5% in accuracy and 1.3% in F1 score compared to the RLM and CLM strategies. While on the Devign and Big-Vul datasets, the average relative improvements are 1.9% and

Table 5
Performance of CAPTURE adopting different types of input.

Dataset	Input	Metrics			
		Acc	F1	Prec	Rec
D2A	CAPTURE_{token}	56.88	47.44	64.09	37.66
	CAPTURE_{ast}	61.24	58.38	65.59	52.60
	CAPTURE_{xsbt}	60.57	61.91	61.81	**62.01**
	CAPTURE	**63.93**	**62.08**	**67.95**	57.14
Devign	CAPTURE_{token}	55.83	44.48	**52.48**	38.60
	CAPTURE_{ast}	58.05	43.21	52.47	36.73
	CAPTURE_{xsbt}	58.36	52.61	50.45	**54.97**
	CAPTURE	**58.57**	**53.53**	52.20	54.93
Big-Vul	CAPTURE_{token}	93.76	16.29	38.62	10.32
	CAPTURE_{ast}	93.88	23.61	44.40	16.08
	CAPTURE_{xsbt}	94.24	27.04	54.38	17.99
	CAPTURE	**94.74**	**31.36**	**62.86**	**20.89**

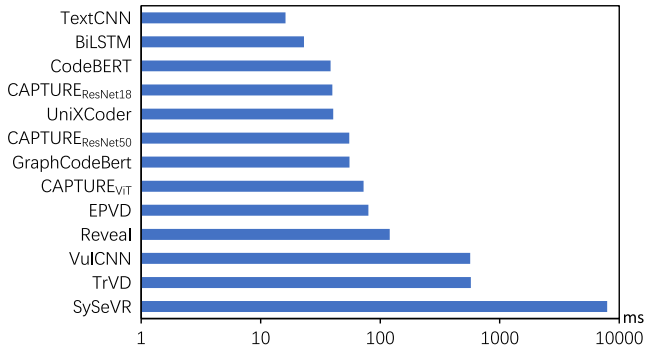

Fig. 5. Runtime overhead comparison.

0.5% in accuracy, and 12.1% and 39.6% in F1 score, respectively. Furthermore, CLM, which determines the label mapping relationships based on the probabilities, outperforms RLM, which uses randomly assigned label mappings.

In addition, as discussed in Section 3.1.2, CAPTURE adopts a mixed sequence representation that conveys both code semantic and structural aspects, for enhancing its capability of comprehending implicit vulnerable patterns. To evaluate this design, we compare its performance against alternative input representations, including (1) the regular code token sequence, (2) the type token sequence derived from the AST that encodes both code structure and element type information, and (3) the X-SBT (Niu et al., 2022) sequence generated by linearizing the AST in a structure-aware manner.

Table 5 summarizes the experimental results. It is evident that substituting the mixed sequence with these input alternatives incurs varying degrees of performance degradation across all datasets. Specifically, these alternatives exhibit average relative accuracy reductions of 6.8%, 2.0%, and 0.8%, and average relative F1 score decreases of 10.0%, 12.8%, and 32.7%, on the D2 A, Devign, and Big-Vul datasts, respectively. Among the alternatives, the X-SBT sequence, which also encodes both code token and structural information, emerges as a competitive option, surpassing the other two alternatives. However, its longer sequence length, which explicitly marks the hierarchy of code elements using XML-style indicators, places it at the disadvantage position compared to our mixed code representation. When the sequence exceeds the maximum size constraint of the model, truncation can lead to the loss of crucial vulnerability information located beyond the truncation point.

Answer to RQ4: The devised DLM remapping strategy plays a key factor in ensuring CAPTURE's outstanding vulnerability detection performance. By integrating label remapping into optimization framework and progressively determining the mapping relationships between the pre-trained model's original labels and the target labels, it effectively enhances the model's reprogramming capabilities. By adopting a mixed sequence representation that encodes both the semantic and structural aspects of the code, CAPTURE enhances its ability to understand vulnerable patterns, leading to improved vulnerability detection capability.

4.2.5. RQ5: Efficiency analysis

In this section, we analyze the training overhead of CAPTURE, which undergoes input preparation and adversarial reprogramming to finish its training. During input preparation, normalized token sequences and ASTs are extracted from the code snippets. In the adversarial reprogramming stage, the primary overheads include forward computations for generating perturbation images, non-linear computations within the ViT model, output label remapping, as well as the backpropagation to update the limited perturbation parameters by minimizing the loss function. To provide an intuitive understanding on the training overhead of CAPTURE and the baseline methods, their average time consumed on training per sample are reported in Fig. 5.

Among all the methods compared, TextCNN and BiLSTM are the most efficient, taking an average of 16 ms and 23 ms per function, respectively, due to their use of simple lexical parsing for code tokens and shallow neural networks for feature encoding. However, their detection accuracy is the lowest due to their limited feature distillation capabilities. In contrast, SySeVR is the most resource-intensive model, with an average processing time of 7910 ms per sample, due to its time-consuming code slicing enforced on the PDG. VulCNN, which also operates on the PDG but computes node centralities and weighted node embeddings to be mapped into images, and the tree-based TrVD, which employs AttRvNN to recursively encode and aggregate the node features from the AST, incur a considerable amount processing time of 567 ms and 574 ms per sample, respectively. The graph-based Reveal, which uses GGNN for feature extraction from the CPG, the EPVD that decomposes syntax-CFG into multiple paths and leverages CodeBERT and CNN for feature extraction, and the fine-tuning CodeBERT, UniXCoder, and GraphCodeBERT, exhibit relatively moderate training overheads, with their times consumed per sample ranging between 39 ms and 120 ms.

In contrast, CAPTURE strikes an optimal balance between vulnerability detection capability and efficiency, averaging 73 ms per sample using the ViT backbone. This overhead is further reduced to 40 ms and 55 ms when smaller backbones like ResNet-18 and ResNet-50 are used, respectively. This makes CAPTURE cost-efficient and highly deployable for practical vulnerability detection. To provide a more comprehensive perspective, Fig. 6 simultaneously visualizes the models' detection effectiveness, as reflected in the average accuracy and F1 scores across the datasets, against their efficiency, which is measured by sample processing speed.

Answer to RQ5: CAPTURE achieves a superior balance between efficiency and effectiveness compared to the other DL-based baselines. By updating only the limited perturbation parameters while leaving the pre-trained model's parameters untouched during model training, it significantly reduces the number of trainable parameters while leading to improved vulnerability detection performance.

5. Discussion

As a DL-based detector, CAPTURE adversarially reprograms pre-trained deep vision neural networks to adapt to the task of detecting vulnerabilities in code snippets. However, the inner workings of the model remain opaque, making it difficult to understand why CAPTURE makes specific predictions (Zheng et al., 2023). This lack of

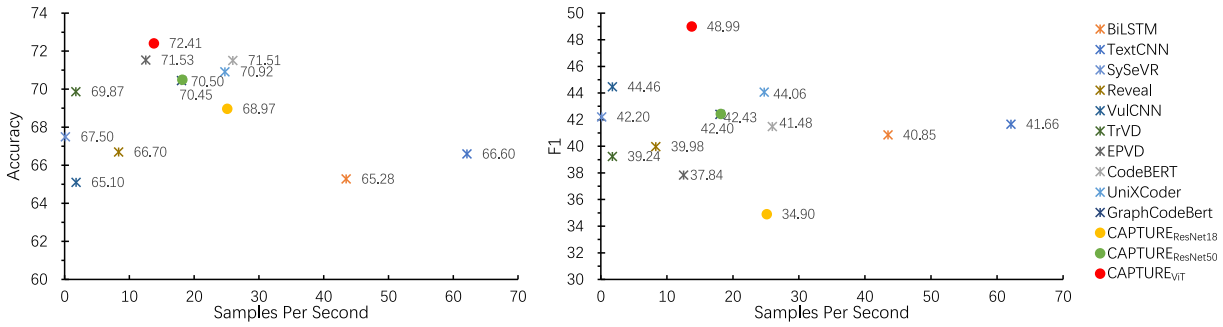

Fig. 6. Comparison of processing speed and accuracy.

interpretability could undermine the model's trustworthiness, limiting its practical adoption. To address this, integrating explainable AI (XAI) (Dwivedi et al., 2023; Cheng et al., 2024b) to pinpoint specific statements (Cao et al., 2024a) or paths (Cheng et al., 2024a) influencing predictions holds promise. One of our future works will focus on developing explainable DL-based vulnerability detection approaches.

Current DL-based methods are limited by sequence length constraints, which can result in missing important vulnerability information that possibly resides beyond the truncation point of the input code snippet. In future work, we aim to explore ways to expand the encoding capacity of adversarial reprogramming models. Promising ways involve converting code into more compact representations using lightweight neural encoding, or reducing the input code size through slicing out less interesting statements, before applying adversarial reprogramming on pre-trained models, so as to ensure a more comprehensive capture of potential weaknesses patterns.

To meet the input requirements of vision models, CAPTURE transforms each token's embedding, retrieved from the perturbation dictionary, into a three-channel image patch. These patches are then arranged sequentially from left to right and top to bottom, maintaining the original token order in the sequence. This ensures that no information is lost during the organization of patches into an image. However, when reshaping a token's d-dimensional embedding into its corresponding $p \times p \times 3$ image patch, there is a potential risk of information loss, particularly if certain features in the original vector exhibit strong spatial or sequential dependencies. Nevertheless, since the perturbation dictionary from which the embeddings are retrieved is learnable, any potential information loss resulting from this vector reshaping can be mitigated or compensated for during the model training process.

Semantics-preserving code transformations (Gao et al., 2023; Qu et al., 2024) may pose a threat to the DL-based vulnerability detectors. Nevertheless, common transformation schemes, such as identifier substitution (Yang et al., 2024a) and light-weight insertion of dead code (Na et al., 2023), which have successfully mislead deep learning models for code summarization (Jha and Reddy, 2023), code completion (Yang et al., 2024b) and functionality classification (Yefet et al., 2020), are unlikely to impact CAPTURE. This attributes to the code normalization performed, which reduces the potential impacts of these types of attacks prior to the code being processed by the prediction model. Nevertheless, more sophisticated strategies, such as embedding specific vulnerabilities into benign samples or injecting substantial amounts of benign yet unreachable code into vulnerable samples, could potentially deceive DL-based detectors. Strengthening CAPTURE's robustness against these potential attacks and conducting systematic evaluation of their impacts on the DL-based vulnerability models are interesting directions that deserve further study.

While our code normalization helps mitigate code transformation attacks, a natural concern arises regarding its potential impact on the semantic comprehension ability of DL-based vulnerability detectors, particularly due to the loss of meaningful variable names during normalization. However, our normalization strategy specifically targets variables and user-defined function names, leaving the names of informative API function calls intact. Thus, we argue that normalization has a limited effect on the model's vulnerability detection capability. This is because vulnerability detectors primarily emphasize identifying indicative vulnerable code patterns, which remain unaffected by normalization, rather than requiring a comprehensive semantic understanding, which is a requirement more relevant to tasks like code summarization (Fang et al., 2024) and code generation (Dong et al., 2024). To substantiate this claim, we evaluated the performance of CAPTURE without code normalization but with comment removal only. Interestingly, the results revealed a slight performance drop. This can be attributed to the increased vocabulary size, where the presence of miscellaneous tokens complicates the effective learning of the perturbation dictionary, distracting the attention of the model from grasping the vulnerable patterns. Additionally, the absence of normalization increases the likelihood of OOV tokens, which negatively impacts the model's inference performance when analyzing unseen test samples.

In this work, CAPTURE is evaluated on code written in C and C++, which are among the programming languages most affected by vulnerabilities. However, in principle, CAPTURE can be easily extended to detect vulnerabilities in code written in other programming languages, provided that the AST can be generated from the target code snippets. The tree-sitter library, which CAPTURE employs to construct ASTs, currently supports robust parsing for over 40 programming languages. As a future work, we plan to extend CAPTURE to support vulnerability detection in other programming languages, such as Java and emerging ones like Solidity (Chen et al., 2024). Furthermore, we aim to explore the potential of adversarial reprogramming to achieve language-agnostic vulnerability prediction, where the model can accept code written in any of the supported languages or even handle a mixture of multiple languages.

6. Related work

This section focuses on discussing two closely related areas of research: the development of advanced DL-based models to improve vulnerability detection capabilities, and data augmentation works aimed at enriching the quantity and diversity of vulnerable samples to enhance model training.

6.1. Deep learning based vulnerability detection

As deep learning models gain popularity for vulnerability detection, numerous methods have emerged to enhance the detection performance. One primary category involves the sequence-based methods (Russell et al., 2018; Ni et al., 2023; Napier et al., 2023), where the input code snippets are typically represented as token sequences through lexical parsing (Hanif and Maffeis, 2022) or as a set of sequences based on heuristic CFG path selection (Zhang et al., 2023b). Neural encoders, such as CNNs (Russell et al., 2018), LSTMs (Lin et al., 2017), or pre-trained language models (Fu and Tantithamthavorn,

2022; Hanif and Maffeis, 2022), are then used to extract features and predict vulnerabilities.

Another prominent line of works is slice-based (Li et al., 2018, 2021a; Zou et al., 2021), where programs are divided into slices relevant to potential vulnerability indicative points, such as calls to certain library or API functions. For instance, SySeVR (Li et al., 2021a) slices the code using vulnerability-related syntax features (*e.g.*, array usage, arithmetic expressions, and pointer operations) and employs a BiGRU model to predict vulnerabilities based on the sliced snippets. However, these methods tend to be less efficient and may overlook vulnerabilities that are not directly associated with these predefined or supposed points of interest.

The more recent works (Chakraborty et al., 2021; Huang et al., 2024; Wen et al., 2023; Zhou et al., 2019) adopt graph-based approaches, leveraging representations such as CPG (Zhou et al., 2019), PDG (Li et al., 2021b), and other graph representations like XFG (Cheng et al., 2021), HeVG (Huang et al., 2024), and MPG (Wen et al., 2024). These methods basically use GNNs to extract higher-level code features for vulnerability prediction. For instance, Reveal (Chakraborty et al., 2021) utilizes GGNN to capture informative features from the CPG, with each node being processed by a GRU while updating the node embeddings via neighborhood aggregation. However, building these overly complex graph structures and training high-performance GNNs is a complicated and time-intensive process, often failing to effectively capture the long-range dependencies within the code graph.

There are also emerging works (Steenhoek et al., 2024; Ding et al., 2024; Nong et al., 2024a; Guo et al., 2024; Fu et al., 2023) that utilize general purpose LLMs, such as ChatGPT and LLaMA, for vulnerability detection using specially designed prompts. Nonetheless, these pioneering efforts demonstrate that LLMs encounter considerable challenges in effectively detecting vulnerabilities, often struggling to understand essential code structures and security-related concepts (Steenhoek et al., 2024). By carefully designing prompts that incorporate chain-of-thought (COT) reasoning (Nong et al., 2024a) and in-context learning, it may be possible to better harness the potential of LLMs for vulnerability detection.

In addition to the majority of works detecting vulnerabilities at the function level, several studies (Li et al., 2021b; Cheng et al., 2024b; Cao et al., 2024a; Cheng et al., 2024a; Cao et al., 2024b; Chu et al., 2024) have targeted more fine-grained statement-level detection to enhance result interpretability. For instance, IVDetect (Li et al., 2021b) employs GNNExplainer to highlight critical statements in the PDG for interpretability. MRC-VulLoc (Tang et al., 2024) integrates a pre-trained model with BiLSTM and CNN to pinpoint vulnerability-triggering paths. MVD (Cao et al., 2022) introduces a flow-sensitive GNN that jointly embeds statement representations and structured information, such as control and data flow, to detect memory-related vulnerabilities at the statement level. VELVET (Ding et al., 2022) identifies vulnerable statements by combining ensemble learning with Transformers for token sequences and GGNNs for code property graphs (CPG). In contrast, our work focuses on addressing different objectives, namely reducing computational costs and achieving robust detection with limited data. Leveraging adversarial reprogramming to extend Capture for locating vulnerable statements offers an intriguing direction for future exploration.

6.2. Data augmentation for enriching vulnerability samples

The limited availability of high-quality vulnerable samples in real-world scenarios significantly hinders the effective training of DL-based vulnerability detectors (Nong et al., 2022; Yang et al., 2023). Consequently, there is increasing interest that generate crafted vulnerable code (Nong et al., 2023) to enhance the learning capabilities of these data-driven models. Nong et al. (2022) investigate leveraging neural code editors to produce vulnerability samples from benign programs. VulGen (Nong et al., 2023) combines pattern mining with deep learning-based localization to generate injection-based vulnerabilities. Similarly, VGX (Nong et al., 2024b) also creates vulnerable samples through vulnerability injection, but it additionally employs data augmentation and manually crafted mutation rules to diversify code edit patterns, with incorporating a Transformer model to identify the contexts for injection. In contrast to these methods that enhance model training through enriched crafted data, Capture employs the naturally data-efficient model adversarial reprogramming to alleviate the data scarcity issue in vulnerability detection.

7. Conclusion

To address the challenges of vulnerable sample scarcity and the high costs associated with training effective DL-based vulnerability detectors, Capture explores a novel solution that leverages cross-modal adversarial reprogramming for vulnerability detection. To ensure the pre-trained vision models built for image classification to be effectively repurposed, some key innovations, including structure- and type-aware AST linearization, ingenious perturbation image generation, and a dual-layer dynamic label remapping scheme, have been devised throughout the learning process. Extensive evaluations across three real-world vulnerability datasets demonstrate that Capture not only achieves superior detection accuracy comparable to SOTA methods but also enhances training efficiency by minimizing the number of parameters needing updates. This makes it particularly advantageous in scenarios with limited available vulnerable samples. As future work, we plan to extend Capture to reprogram more advanced pre-trained models, while further enhancing its effectiveness, explainability, and robustness.

CRediT authorship contribution statement

Zhenzhou Tian: Writing – review & editing, Supervision, Methodology. **Rui Qiu:** Writing – original draft, Methodology, Data curation. **Yudong Teng:** Data curation, Validation. **Jiaze Sun:** Validation. **Yanping Chen:** Methodology. **Lingwei Chen:** Writing – review & editing.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Zhenzhou Tian reports financial support was provided by National Natural Science foundation of china. Zhenzhou Tian reports financial support was provided by Natural Science Basic Research Program of Shaanxi Province. Zhenzhou Tian reports financial support was provided by the Youth innovation Team of Shaanxi Universities. Zhenzhou Tian reports financial support was provided by Key R&D Project of Science and Technology Department of Shaanxi. Zhenzhou Tian reports financial support was provided by Key Industrial Chain Core Technology Research Project of Xi'an. Rui Qiu reports financial support was provided by Graduate Innovation Fund of Xi'an University of Posts and Telecommunications.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (62272387, 61702414), the Natural Science Basic Research Program of Shaanxi (2022JM-342, 2018JQ6078), the Youth Innovation Team of Shaanxi Universities "Industrial Big Data Analysis and Intelligent Processing", the Special Funds for Construction of Key Disciplines in Universities in Shaanxi, the Key R&D Project of Science and Technology Department of Shaanxi (2023-YBGY-030), Key industrial chain Core Technology Research Project of Xi'an (23ZDCYJSGG0028-2022) and Graduate Innovation Fund of Xi'an University of Posts and Telecommunications (CXJJYL2023040).

Data availability

I have shared the link to my code in the manuscript.

References

Anon, 2024. "CVE." [Online]. Available: https://cve.mitre.org/.

Anon, 2024b. Infer: A tool to detect bugs in Java and C/C++/Objective-C code before it ships. https://fbinfer.com.

Baluja, Shumeet, Fischer, Ian, 2018. Learning to attack: Adversarial transformation networks. In: AAAI, Vol. 32, No. 1.

Cao, Sicong, Sun, Xiaobing, Bo, Lili, Wu, Rongxin, Li, Bin, Tao, Chuanqi, 2022. MVD: memory-related vulnerability detection based on flow-sensitive graph neural networks. In: ICSE. pp. 1456–1468.

Cao, Sicong, Sun, Xiaobing, Wu, Xiaoxue, Lo, David, Bo, Lili, Li, Bin, Liu, Wei, 2024a. Coca: Improving and explaining graph neural network-based vulnerability detection systems. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24, Association for Computing Machinery, New York, NY, USA.

Cao, Sicong, Sun, Xiaobing, Wu, Xiaoxue, Lo, David, Bo, Lili, Li, Bin, Liu, Wei, 2024b. Coca: Improving and explaining graph neural network-based vulnerability detection systems. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24, Association for Computing Machinery, New York, NY, USA.

Chakraborty, Saikat, Krishna, Rahul, Ding, Yangruibo, Ray, Baishakhi, 2021. Deep learning based vulnerability detection: Are we there yet. TSE 3280–3296.

Chen, Pin-Yu, 2024. Model reprogramming: Resource-efficient cross-domain machine learning. In: AAAI.

Chen, Lingwei, Fan, Yujie, Ye, Yanfang, 2021. Adversarial reprogramming of pretrained neural networks for fraud detection. In: CIKM. pp. 2935–2939.

Chen, Lingwei, Li, Xiaoting, Wu, Dinghao, 2022. Adversarially reprogramming pre-trained neural networks for data-limited and cost-efficient malware detection. In: Proceedings of the 2022 SIAM International Conference on Data Mining. SDM, SIAM, pp. 693–701.

Chen, Yizhou, Sun, Zeyu, Gong, Zhihao, Hao, Dan, 2024. Improving smart contract security with contrastive learning-based vulnerability detection. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24, Association for Computing Machinery, New York, NY, USA.

Chen, Aochuan, Yao, Yuguang, Chen, Pin-Yu, Zhang, Yihua, Liu, Sijia, 2023. Understanding and improving visual prompting: A label-mapping perspective. In: CVPR. pp. 19133–19143.

Cheng, Xiao, Nie, Xu, Li, Ningke, Wang, Haoyu, Zheng, Zheng, Sui, Yulei, 2024a. How about bug-triggering paths?-understanding and characterizing learning-based vulnerability detectors. TDSC 21 (2), 542–558.

Cheng, Xiao, Wang, Haoyu, et al., 2021. Deepwukong: Statically detecting software vulnerabilities using deep graph neural network. ToSEM 30 (3), 1–33.

Cheng, Baijun, Zhao, Mingsheng, Wang, Kailong, Wang, Meizhen, Bai, Guangdong, Feng, Ruitao, et al., 2024b. Beyond fidelity: Explaining vulnerability localization of learning-based detectors. ToSEM.

Chu, Zhaoyang, Wan, Yao, Li, Qian, Wu, Yang, Zhang, Hongyu, Sui, Yulei, Xu, Guandong, Jin, Hai, 2024. Graph neural networks for vulnerability detection: A counterfactual explanation. In: ISSTA. Association for Computing Machinery, New York, NY, USA, pp. 389–401.

Ding, Yangruibo, Fu, Yanjun, Ibrahim, Omniyyah, Sitawarin, Chawin, Chen, Xinyun, Alomair, Basel, Wagner, David, Ray, Baishakhi, Chen, Yizheng, 2024. Vulnerability detection with code language models: How far are we?. URL https://arxiv.org/abs/2403.18624.

Ding, Yangruibo, Suneja, Sahil, Zheng, Yunhui, Laredo, Jim, Morari, Alessandro, Kaiser, Gail, Ray, Baishakhi, 2022. VELVET: a noVel ensemble learning approach to automatically locate VulnErable sTatements. In: SANER. pp. 959–970.

Dong, Yihong, Jiang, Xue, Jin, Zhi, Li, Ge, 2024. Self-collaboration code generation via ChatGPT. ACM Trans. Softw. Eng. Methodol. 33 (7).

Dosovitskiy, Alexey, Beyer, Lucas, Kolesnikov, Alexander, Weissenborn, Dirk, Zhai, Xiaohua, Unterthiner, Thomas, Dehghani, Mostafa, Minderer, Matthias, Heigold, Georg, Gelly, Sylvain, Uszkoreit, Jakob, Houlsby, Neil, 2021. An image is worth 16 × 16 words: Transformers for image recognition at scale. In: ICLR.

Dwivedi, Rudresh, Dave, Devam, Naik, Het, Singhal, Smiti, Omer, Rana, Patel, Pankesh, Qian, Bin, Wen, Zhenyu, Shah, Tejal, Morgan, Graham, Ranjan, Rajiv, 2023. Explainable AI (XAI): Core ideas, techniques, and solutions. ACM Comput. Surv. 55 (9).

Elsayed, Gamaleldin F., Goodfellow, Ian, Sohl-Dickstein, Jascha, 2018. Adversarial reprogramming of neural networks. arXiv.

Fan, Jiahao, Li, Yi, Wang, Shaohua, Nguyen, Tien N., 2020. A C/C++ code vulnerability dataset with code changes and CVE summaries. In: Proceedings of the 17th International Conference on Mining Software Repositories. MSR '20, Association for Computing Machinery, New York, NY, USA, pp. 508–512.

Fang, Chunrong, Sun, Weisong, Chen, Yuchen, Chen, Xiao, Wei, Zhao, Zhang, Quanjun, You, Yudu, Luo, Bin, Liu, Yang, Chen, Zhenyu, 2024. Esale: Enhancing code-summary alignment learning for source code summarization. IEEE Trans. Softw. Eng. 50 (8), 2077–2095.

Feng, Zhangyin, Guo, Daya, Tang, Duyu, Duan, Nan, Feng, Xiaocheng, Gong, Ming, Shou, Linjun, et al., 2020. Codebert: A pre-trained model for programming and natural languages. arXiv.

Fu, Michael, Tantithamthavorn, Chakkrit, 2022. Linevul: A transformer-based line-level vulnerability prediction. In: Proceedings of the 19th International Conference on Mining Software Repositories. pp. 608–620.

Fu, Michael, Tantithamthavorn, Chakkrit Kla, Nguyen, Van, Le, Trung, 2023. ChatGPT for vulnerability detection, classification, and repair: How far are we? In: 2023 30th Asia-Pacific Software Engineering Conference. APSEC, pp. 632–636.

Gao, Fengjuan, Wang, Yu, Wang, Ke, 2023. Discrete adversarial attack to models of code. Proc. ACM Program. Lang. 7 (PLDI).

Goodfellow, Ian J., Shlens, Jonathon, Szegedy, Christian, 2014. Explaining and harnessing adversarial examples. arXiv.

Guo, Daya, Lu, Shuai, Duan, Nan, Wang, Yanlin, Zhou, Ming, Yin, Jian, 2022. UniXcoder: Unified cross-modal pre-training for code representation. In: ACL. pp. 7212–7225.

Guo, Yuejun, Patsakis, Constantinos, Hu, Qiang, Tang, Qiang, Casino, Fran, 2024. Outside the comfort zone: Analysing LLM capabilities in software vulnerability detection. In: Garcia-Alfaro, Joaquin, Kozik, Rafał, Choraś, Michał, Katsikas, Sokratis (Eds.), Computer Security – ESORICS 2024. Springer Nature Switzerland, Cham, pp. 271–289.

Guo, Daya, Ren, Shuo, Lu, Shuai, Feng, Zhangyin, Tang, Duyu, LIU, Shujie, Zhou, Long, Duan, Nan, Svyatkovskiy, Alexey, Fu, Shengyu, Tufano, Michele, Deng, Shao Kun, Clement, Colin, Drain, Dawn, Sundaresan, Neel, Yin, Jian, Jiang, Daxin, Zhou, Ming, 2021. GraphCode{bert}: Pre-training code representations with data flow. In: International Conference on Learning Representations.

Hambardzumyan, Karen, Khachatrian, Hrant, May, Jonathan, 2021. Warp: Word-level adversarial reprogramming. arXiv preprint arXiv:2101.00121.

Hanif, Hazim, Maffeis, Sergio, 2022. Vulberta: Simplified source code pre-training for vulnerability detection. In: IJCNN. IEEE, pp. 1–8.

He, Kaiming, Zhang, Xiangyu, Ren, Shaoqing, Sun, Jian, 2016. Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 770–778.

Hu, Tiancheng, Xu, Zijing, Fang, Yilin, Wu, Yueming, Yuan, Bin, Zou, Deqing, Jin, Hai, 2023. Fine-grained code clone detection with block-based splitting of abstract syntax tree. In: ISSTA. In: ISSTA 2023, Association for Computing Machinery, New York, NY, USA, pp. 89–100.

Huang, Yuanming, He, Mingshu, Wang, Xiaojuan, Zhang, Jie, 2024. HeVulD: A static vulnerability detection method using heterogeneous graph code representation. IEEE Trans. Inf. Forensics Secur. 1.

Iandola, Forrest, Moskewicz, Matt, Karayev, Sergey, Girshick, Ross, Darrell, Trevor, Keutzer, Kurt, 2014. Densenet: Implementing efficient convnet descriptor pyramids. arXiv preprint arXiv:1404.1869.

Jha, Akshita, Reddy, Chandan K., 2023. CodeAttack: Code-based adversarial attacks for pre-trained programming language models. AAAI 37 (12), 14892–14900.

Kang, Wooseok, Son, Byoungho, Heo, Kihong, 2022. TRACER: Signature-based static analysis for detecting recurring vulnerabilities. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. CCS '22, Association for Computing Machinery, New York, NY, USA, pp. 1695–1708.

Li, Jian, He, Pinjia, Zhu, Jieming, Lyu, Michael R., 2017. Software defect prediction via convolutional neural network. In: QRS. IEEE, pp. 318–328.

Li, Zhen, Zou, Deqing, Xu, Shouhuai, Jin, Hai, Zhu, Yawei, Chen, Zhaoxuan, 2021a. Sysevr: A framework for using deep learning to detect software vulnerabilities. TDSC 19 (4), 2244–2258.

Li, Zhen, Zou, Deqing, Xu, Shouhuai, Ou, Xinyu, Jin, Hai, Wang, Sujuan, Deng, Zhijun, Zhong, Yuyi, 2018. Vuldeepecker: A deep learning-based system for vulnerability detection. arXiv preprint arXiv:1801.01681.

Li, Yi, et al., 2021b. Vulnerability detection with fine-grained interpretations. In: ESEC/FSE. pp. 292–303.

Liang, Hongshuo, He, Erlu, Zhao, Yangyang, Jia, Zhe, Li, Hao, 2022. Adversarial attack and defense: A survey. Electronics 11 (8), 1283.

Lin, Guanjun, Zhang, Jun, Luo, Wei, Pan, Lei, Xiang, Yang, 2017. POSTER: Vulnerability discovery with function representation learning from unlabeled projects. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. pp. 2539–2541.

Lipp, Stephan, Banescu, Sebastian, Pretschner, Alexander, 2022. An empirical study on the effectiveness of static C code analyzers for vulnerability detection. In: ISSTA 2022, Association for Computing Machinery, New York, NY, USA, pp. 544–555.

Madry, Aleksander, 2017. Towards deep learning models resistant to adversarial attacks. arXiv preprint arXiv:1706.06083.

Moosavi-Dezfooli, Seyed-Mohsen, Fawzi, Alhussein, Fawzi, Omar, Frossard, Pascal, 2017. Universal adversarial perturbations. In: CVPR. pp. 1765–1773.

Moosavi-Dezfooli, Seyed-Mohsen, Fawzi, Alhussein, Frossard, Pascal, 2016. Deepfool: a simple and accurate method to fool deep neural networks. In: CVPR. pp. 2574–2582.

Na, CheolWon, Choi, YunSeok, Lee, Jee-Hyong, 2023. DIP: Dead code insertion based black-box attack for programming language model. In: Rogers, Anna, Boyd-Graber, Jordan, Okazaki, Naoaki (Eds.), ACL. Toronto, Canada, pp. 7777–7791.

Napier, Kollin, Bhowmik, Tanmay, Wang, Shaowei, 2023. An empirical study of text-based machine learning models for vulnerability detection. Empir. Softw. Eng. 28 (2), 38.

Neekhara, Paarth, Hussain, Shehzeen, Du, Jinglong, Dubnov, Shlomo, Koushanfar, Farinaz, McAuley, Julian, 2022. Cross-modal adversarial reprogramming. In: WACV. pp. 2427–2435.

Neekhara, Paarth, Hussain, Shehzeen, Dubnov, Shlomo, Koushanfar, Farinaz, 2018. Adversarial reprogramming of text classification neural networks. arXiv preprint arXiv:1809.01829.

Ni, Chao, Yin, Xin, Yang, Kaiwen, Zhao, Dehai, Xing, Zhenchang, Xia, Xin, 2023. Distinguishing look-alike innocent and vulnerable code by subtle semantic representation learning and explanation. In: FSE. In: ESEC/FSE 2023, Association for Computing Machinery, New York, NY, USA, pp. 1611–1622.

Niu, Changan, Li, Chuanyi, Ng, Vincent, Ge, Jidong, Huang, Liguo, Luo, Bin, 2022. SPT-code: Sequence-to-sequence pre-training for learning source code representations. In: ICSE.

Nong, Yu, Aldeen, Mohammed, Cheng, Long, Hu, Hongxin, Chen, Feng, Cai, Haipeng, 2024a. Chain-of-thought prompting of large language models for discovering and fixing software vulnerabilities.

Nong, Yu, Fang, Richard, et al., 2024b. VGX: Large-scale sample generation for boosting learning-based software vulnerability analyses. In: ICSE.

Nong, Yu, Ou, Yuzhe, Pradel, Michael, Chen, Feng, Cai, Haipeng, 2022. Generating realistic vulnerabilities via neural code editing: an empirical study. In: FSE. pp. 1097–1109.

Nong, Yu, Ou, Yuzhe, Pradel, Michael, Chen, Feng, Cai, Haipeng, 2023. VULGEN: Realistic vulnerability generation via pattern mining and deep learning. In: ICSE.

Nong, Yu, Yang, Haoran, Chen, Feng, Cai, Haipeng, 2024c. VinJ: An automated tool for large-scale software vulnerability data generation. In: FSE. NY, USA, pp. 567–571.

Pagliardini, Matteo, Gupta, Prakhar, Jaggi, Martin, 2018. Unsupervised learning of sentence embeddings using compositional n-gram features. In: Walker, Marilyn, Ji, Heng, Stent, Amanda (Eds.), Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers). Association for Computational Linguistics, New Orleans, Louisiana, pp. 528–540.

Perl, Henning, Dechand, Sergej, Smith, Matthew, Arp, Daniel, Yamaguchi, Fabian, Rieck, Konrad, Fahl, Sascha, Acar, Yasemin, 2015. Vccfinder: Finding potential vulnerabilities in open-source projects to assist code audits. In: CCS. pp. 426–437.

Qian, Hanwei, Liu, Wei, Ding, Ziqi, Sun, Weisong, Fang, Chunrong, 2023. Abstract syntax tree for method name prediction: How far are we? In: 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security. QRS, pp. 464–475.

Qu, Yubin, Huang, Song, Yao, Yongming, 2024. A survey on robustness attacks for deep code models. Autom. Softw. Eng. 31 (2), 65, URL DOI: 10.1007/s10515-024-00464-7.

Russell, Rebecca, Kim, Louis, et al., 2018. Automated vulnerability detection in source code using deep representation learning. In: ICMLA. pp. 757–762.

Steenhoek, Benjamin, Rahman, Md Mahbubur, Jiles, Richard, Le, Wei, 2023. An empirical study of deep learning models for vulnerability detection. In: ICSE. pp. 2237–2248.

Steenhoek, Benjamin, Rahman, Md Mahbubur, Roy, Monoshi Kumar, Alam, Mirza Sanjida, Barr, Earl T., Le, Wei, 2024. A comprehensive study of the capabilities of large language models for vulnerability detection.

Szegedy, Christian, Vanhoucke, Vincent, Ioffe, Sergey, Shlens, Jon, Wojna, Zbigniew, 2016. Rethinking the inception architecture for computer vision. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 2818–2826.

Tang, Gaigai, Yang, Lin, Zhang, Long, Kuang, Hongyu, Wang, Huiqiang, 2024. MRC-VulLoc: Software source code vulnerability localization based on multi-choice reading comprehension. Comput. Secur. 141, 103816.

Tian, Zhenzhou, Tian, Binhui, Lv, Jiajun, Chen, Yanping, Chen, Lingwei, 2024. Enhancing vulnerability detection via AST decomposition and neural sub-tree encoding. ESWA 238, 121865.

Tsai, Yun-Yun, Chen, Pin-Yu, Ho, Tsung-Yi, 2020. Transfer learning without knowing: Reprogramming black-box machine learning models with scarce data and limited resources. In: International Conference on Machine Learning. PMLR, pp. 9614–9624.

Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Łukasz, Polosukhin, Illia, 2017. Attention is all you need. In: NeurIPS. pp. 5998–6008.

Wen, Xin-Cheng, Chen, Yupan, Gao, Cuiyun, Zhang, Hongyu, Zhang, Jie M., Liao, Qing, 2023. Vulnerability detection with graph simplification and enhanced graph representation learning. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, pp. 2275–2286.

Wen, Xin-Cheng, Gao, Cuiyun, Ye, Jiaxin, Li, Yichen, Tian, Zhihong, Jia, Yan, Wang, Xuan, 2024. Meta-path based attentional graph learning model for vulnerability detection. IEEE Trans. Softw. Eng. 50 (3), 360–375.

Wu, Yueming, Zou, Deqing, Dou, Shihan, Yang, Wei, Xu, Duo, Jin, Hai, 2022. VulCNN: An image-inspired scalable vulnerability detection system. In: ICSE 2022. pp. 2365–2376.

Xu, Aidong, Dai, Tao, Chen, Huajun, Ming, Zhe, Li, Weining, 2018. Vulnerability detection for source code using contextual LSTM. In: ICSAI. IEEE, pp. 1225–1230.

Yang, Yulong, Fan, Haoran, Lin, Chenhao, Li, Qian, Zhao, Zhengyu, Shen, Chao, 2024a. Exploiting the adversarial example vulnerability of transfer learning of source code. IEEE Trans. Inf. Forensics Secur. 19, 5880–5894.

Yang, Xu, Wang, Shaowei, Li, Yi, Wang, Shaohua, 2023. Does data sampling improve deep learning-based vulnerability detection? Yeas! and Nays!. In: ICSE. pp. 2287–2298.

Yang, Guang, Zhou, Yu, Yang, Wenhua, Yue, Tao, Chen, Xiang, Chen, Taolue, 2024b. How important are good method names in neural code generation? A model robustness perspective. ACM Trans. Softw. Eng. Methodol. 33 (3).

Yefet, Noam, Alon, Uri, Yahav, Eran, 2020. Adversarial examples for models of code. Proc. ACM Program. Lang. 4 (OOPSLA).

Yuan, Xiaoyong, He, Pan, Zhu, Qile, Li, Xiaolin, 2019. Adversarial examples: Attacks and defenses for deep learning. IEEE Trans. Neural Networks Learn. Syst. 30 (9), 2805–2824.

Zhang, Jianping, Huang, Yizhan, Wu, Weibin, Lyu, Michael R., 2023a. Transferable adversarial attacks on vision transformers with token gradient regularization. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. CVPR, pp. 16415–16424.

Zhang, Junwei, Liu, Zhongxin, et al., 2023b. Vulnerability detection by learning from syntax-based execution paths of code. TSE 49 (8), 4196–4212.

Zheng, Yang, Feng, Xiaoyi, Xia, Zhaoqiang, Jiang, Xiaoyue, Demontis, Ambra, Pintor, Maura, Biggio, Battista, Roli, Fabio, 2023. Why adversarial reprogramming works, when it fails, and how to tell the difference. Inform. Sci. 632, 130–143.

Zheng, Yunhui, Pujar, Saurabh, Lewis, Burn, Buratti, Luca, Epstein, Edward, Yang, Bo, Laredo, Jim, Morari, Alessandro, Su, Zhong, 2021. D2a: A dataset built for ai-based vulnerability detection methods using differential analysis. In: ICSE-SEIP. pp. 111–120.

Zhou, Yaqin, Liu, Shangqing, Siow, Jingkai, Du, Xiaoning, Liu, Yang, 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. NeurIPS 32.

Zou, Deqing, Wang, Sujuan, Xu, Shouhuai, Li, Zhen, Jin, Hai, 2021. muVulDeePecker: A deep learning-based system for multiclass vulnerability detection. TDSC 2224–2236.